Panduan Belajar Kode TugasKu

> Tujuan: kamu **paham setiap baris**, bisa **menjelaskan alurnya**, dan bisa **menjawab pertanyaan penguji** tanpa contekan. Cara pakai: ikuti urutan tahap 0 → 9. Jangan loncat. Setiap tahap ada **tujuan, yang dibaca, penjelasan, latihan, dan cek paham**.

Peta Belajar (7 hari)

Hari	Tahap	Fokus	Waktu
1	0 – 1	Gambaran besar + dasar JavaScript yang dipakai	1,5 jam
2	2 – 3	HTML & CSS	1,5 jam
3	4 – 5	`firebase.js` + `auth.js` (login & register)	1,5 jam
4	6 (bagian 1–6)	`app.js` : cek login, logout, CRUD, toggle	2 jam
5	6 (bagian 7–10) + 7	`app.js` : filter, statistik, render + Security Rules	2 jam
6	8	Latihan modifikasi kode (uji pemahaman)	2 jam
7	9	Latihan presentasi + kuis + ketik ulang dari nol	2 jam

Tips: buka file kodenya di VS Code di sebelah panduan ini, dan jalankan website lewat Live Server sambil belajar. Setiap kali paham satu bagian, **jelaskan dengan suaramu sendiri** (boleh rekam di HP).

TAHAP 0 — Gambaran Besar (wajib hafal)

0.1 Isi project

```
todolist/
├── index.html        → halaman LOGIN
├── register.html     → halaman DAFTAR
├── dashboard.html    → halaman UTAMA (kelola tugas)
├── css/style.css     → semua tampilan
├── js/firebase.js    → koneksi ke Firebase
├── js/auth.js        → logika login & daftar
├── js/app.js         → logika dashboard
└── firestore.rules   → aturan keamanan database
```

0.2 Pembagian tugas tiga bahasa

Bahasa	Analogi	Tugasnya
HTML	Kerangka rumah	Menentukan **ada apa saja** di halaman (form, tombol, kotak)
CSS	Cat & dekorasi	Menentukan **tampilannya** (warna, ukuran, posisi)
JavaScript	Listrik & saklar	Menentukan **apa yang terjadi** saat diklik / diisi
Firebase	Gudang + satpam di internet	Menyimpan data (Firestore) dan mengurus akun (Authentication)

0.3 Alur aplikasi (gambar ini di kertas sampai hafal)



0.4 Kalimat ringkas untuk penguji (hafalkan)

> *"Aplikasi saya terdiri dari 3 halaman: login, daftar, dan dashboard. HTML untuk struktur, CSS untuk tampilan, JavaScript untuk logika. Data disimpan di Cloud Firestore, dan akun dikelola Firebase Authentication. Di dashboard user bisa menambah, melihat, mengedit, menghapus, dan menandai tugas selesai, serta mencari, memfilter, dan melihat statistik."*

✅ **Cek paham:** Bisakah kamu menggambar alur 0.3 tanpa melihat?

TAHAP 1 — Dasar JavaScript yang Dipakai di Project Ini

Kamu **tidak perlu** belajar semua JavaScript. Cukup konsep di bawah — semuanya muncul di kode.

Konsep	Contoh di kode	Artinya
`const` / `let`	`let editId = null;`	`const` = nilai tetap, `let` = nilai bisa berubah
String	`"Halo, " + nama`	Teks; `+` untuk menyambung
Boolean	`selesai: false`	Hanya `true` / `false`
Object	`{ judul: "...", prioritas: "tinggi" }`	Kumpulan data berpasangan **nama: nilai**
Array	`semuaTugas = [ {...}, {...} ]`	Daftar banyak data
Function	`function resetForm() { ... }`	Kumpulan perintah yang bisa dipanggil berulang
Arrow function	`(t) ⇒ t.selesai`	Cara singkat menulis function
if / else	`if (judul === "") { ... }`	Percabangan / pengambilan keputusan
`===` / `!==`	`a.selesai !== b.selesai`	Sama persis / tidak sama
`!`	`!statusSekarang`	Membalik nilai: `true` ↔ `false`
`&&` / `||`	`!t.selesai && t.deadline`	DAN / ATAU
Ternary	`t.selesai ? "✓" : ""`	If-else satu baris: `kondisi ? jikaBenar : jikaSalah`
`null`	`let editId = null;`	"Kosong / belum ada isinya"
`for`	`for (let i = 0; i < potongan.length; i++)`	Mengulang perintah sebanyak jumlah tertentu
`return`	`return false;`	Keluar dari function sekaligus mengirim hasil

Method array (penting untuk filter & statistik)

```
const angka = [5, 2, 8];
angka.filter(x ⇒ x > 3);    // [5, 8]     → saring yang memenuhi syarat
angka.map(x ⇒ x * 2);       // [10, 4, 16] → ubah setiap isi
angka.find(x ⇒ x > 3);      // 5          → ambil SATU yang pertama cocok
angka.forEach(x ⇒ ...);     // lakukan sesuatu untuk setiap isi
angka.sort((a, b) ⇒ a - b); // [2, 5, 8]  → urutkan
angka.length;               // 3          → jumlah isi
```

DOM (cara JavaScript mengubah halaman)

```
document.getElementById("judul")      // ambil elemen dengan id="judul"
elemen.value                          // isi dari input
elemen.textContent = "Halo"           // ganti teks elemen
elemen.addEventListener("click", fn)  // jalankan fn saat diklik
document.createElement("div")         // buat elemen baru
induk.appendChild(anak)               // masukkan elemen ke dalam elemen lain
elemen.classList.add("x") / remove    // tambah / hapus class CSS
```

async / await (penting untuk Firebase)

Mengambil data dari internet butuh waktu. `await` artinya **"tunggu sampai selesai, baru lanjut"**. Function yang memakai `await` harus diberi kata `async`.

```
async function contoh() {
  await simpanKeFirebase();    // tunggu dulu
  tampilkanPesan("Berhasil");  // baru jalan setelah simpan selesai
}
```

`try { ... } catch (err) { ... }` → jika ada error di dalam `try`, program lompat ke `catch` (tidak crash).

import / export

```
// firebase.js
export const db = ...;              // "meminjamkan" db ke file lain
// app.js
import { db } from "./firebase.js";  // "meminjam" db dari firebase.js
```

Karena memakai import/export, di HTML ditulis `<script type="module">` dan harus dibuka lewat **Live Server**.

🧪 **Latihan:** buka Console browser (F12 → Console), ketik dan jalankan contoh method array di atas. ✅ **Cek paham:** Apa beda `filter` dan `find`? Apa fungsi `await`?

TAHAP 2 — HTML

2.1 `index.html` (login)

Baca filenya, perhatikan:

- `<head>` : `meta viewport` (agar pas di HP), link Google Font **Plus Jakarta Sans** (font buatan studio Indonesia, Tokotype), link `css/style.css`.
- `<form id="formLogin">` berisi input `email` dan `password` dengan atribut `required` (tidak boleh kosong).
- `<p id="pesan">` → tempat menampilkan pesan error dari JavaScript.
- `<script type="module" src="js/auth.js">` → menghubungkan ke logika login.

2.2 `register.html`

Sama seperti login, ditambah input `nama` dan `konfirmasi`. `minlength="6"` = password minimal 6 karakter (aturan Firebase). **Kedua halaman memakai `auth.js` yang sama** — nanti JS mengecek form mana yang ada.

2.3 `dashboard.html`

Bagi menjadi 5 blok:

1. **Header** — logo, `#sapaan` ("Halo, nama"), tombol `#btnLogout`.
2. **Ringkasan** `.ringkasan` — progress bar `#progressBar` + persen `#persenText`, lalu 4 angka statistik `statTotal`, `statSelesai`, `statBelum`, `statTerlambat` dalam `<dl>` (daftar istilah: `<dt>` = nama, `<dd>` = nilainya).
3. **Form tugas** `#formTugas` — judul, deskripsi, prioritas, kategori, deadline, tombol `#btnSimpan` & `#btnBatal`.
4. **Toolbar** — `#inputCari`, `#filterStatus`, `#filterKategori`, `#urutkan`.
5. **Daftar** `#daftarTugas` dan **paginasi** `#paginasi` — **kosong di HTML**, diisi oleh JavaScript.

Lalu `#toast` (kotak notifikasi) dan `<script type="module" src="js/app.js">`.

Kunci penting: setiap elemen yang dipakai JS punya `id`. Hubungan HTML ↔ JS terjadi lewat `document.getElementById("id")`.

🍼 **Latihan:** cari di `app.js` semua `getElementById(...)`, lalu cocokkan dengan id di `dashboard.html`. ✅ **Cek paham:** Kenapa `#daftarTugas` kosong di HTML?

TAHAP 3 — CSS

Tidak perlu hafal semua baris. Pahami **6 konsep** ini:

Konsep	Di mana	Penjelasan
Variabel CSS	`:root { --tinta: #1e40af; }`	Warna disimpan di satu tempat, dipakai dengan `var(--tinta)`. Ganti sekali, berubah semua
Warna transparan	`rgba(255, 255, 255, .62)`	Putih dengan kejernihan 62% — dasar efek **kaca**
Backdrop filter	`backdrop-filter: blur(20px)`	Mengaburkan apa pun **di belakang** panel, sehingga panel terlihat seperti kaca buram
Pseudo-element	`body::before`, `body::after`	Membuat "elemen tambahan" lewat CSS saja — dipakai untuk menggambar **bulatan cahaya pagi** di latar
Filter blur	`filter: blur(70px)`	Mengaburkan elemen itu sendiri — bulatan warna jadi cahaya lembut
Flexbox	`.topbar`, `.tugas`, `.auth-page`	Menyusun elemen sebaris/sekolom dan menengahkan (`justify-content`, `align-items`)
Grid	`.stats`, `.grid-utama`, `.toolbar`	Membuat kolom: `repeat(4, 1fr)` = 4 kolom sama lebar
Class status	`.selesai`, `.tersembunyi`, `.memuat`	JS menambah/menghapus class → tampilan berubah (dicoret, disembunyikan)
Transition	`.btn`, `.progress-bar`	Animasi halus saat berubah
Media query	`@media (max-width: 640px)`	Aturan khusus layar kecil (HP): kartu jadi 2 kolom

Contoh hubungan JS ↔ CSS:

```
.tugas.selesai h3 { text-decoration: line-through; }   /* judul dicoret */
```

```
item.className = "tugas" + (t.selesai ? " selesai" : "");  // JS memberi class "selesai"
```

Konsep desain glassmorphism "kaca jendela kelas di pagi hari" (bagus diceritakan ke penguji):

- **Glassmorphism** = gaya desain yang membuat panel terlihat seperti kaca buram: latar putih transparan (`rgba`), latar di belakangnya dikaburkan (`backdrop-filter: blur`), dan

tepi putih tipis seperti pantulan cahaya.

- Di belakang kaca ada tiga **cahaya pagi**: biru langit (`--langit`), peach matahari (`--pagi`), dan hijau daun (`--daun`), dibuat dari `body::before` dan `body::after` yang di-blur.
- Hanya ada tiga tingkat kaca: panel (`--kaca`), input dan baris tugas saat di-hover (`--kaca-tipis`), dan tugas yang **sedang dikerjakan** dibuat kaca paling terang dengan tepi biru supaya menonjol.
- Warna utama `--tinta` biru, tugas selesai **dicoret**, dan `--stabilo` kuning untuk tanda "Hari ini".
- Aksesibilitas:
  - Fokus keyboard terlihat (`:focus-visible`).
  - Browser yang tidak mendukung blur memakai kaca yang lebih solid (`@supports not (backdrop-filter: ...)`), supaya teks tetap terbaca.
  - Pengguna yang memilih *kurangi transparansi* mendapat panel putih solid (`@media (prefers-reduced-transparency: reduce)`).
  - Animasi dimatikan untuk pengguna yang memilih *reduce motion* (`@media (prefers-reduced-motion)`).

🧪 **Latihan:** ganti `--tinta` menjadi `#0f766e` , simpan, lihat seluruh warna utama berubah. Kembalikan lagi. ✅ **Cek paham:** Apa fungsi `@media` ? Bagaimana tugas selesai bisa tercoret?

TAHAP 4 — `js/firebase.js` (koneksi)

Baca file. Hanya 4 langkah:

1. `import` tiga fungsi dari Firebase lewat link CDN (`initializeApp` , `getAuth` , `getFirestore`).
2. `firebaseConfig` = "alamat & identitas" project Firebase kita (dari Firebase Console).
3. `initializeApp(firebaseConfig)` = menyambungkan website ke project Firebase.
4. `export const auth` dan `export const db` → dipinjamkan ke `auth.js` dan `app.js` .

Pertanyaan jebakan: *"apiKey kok terlihat, apa tidak bahaya?" → "apiKey Firebase memang publik, hanya identitas project. Yang melindungi data adalah Security Rules di Firestore."*

✅ **Cek paham:** Kenapa koneksi dibuat di file terpisah? (Supaya ditulis sekali, dipakai di banyak file.)

TAHAP 5 — `js/auth.js` (login & daftar)

Baca dari atas ke bawah, ikuti urutan ini:

5.1 Ambil elemen

`pesan`, `formLogin`, dan `formRegister` diambil dengan `document.getElementById`. File ini dipakai **dua halaman**: di halaman login `formRegister` bernilai `null` (tidak ada), di halaman daftar `formLogin` yang `null`. Karena itu kodenya dibungkus `if (formLogin ≠ null)` dan `if (formRegister ≠ null)`.

5.2 Sudah login? Langsung ke dashboard

```
onAuthStateChanged(auth, (user) ⇒ {
  if (user ≠ null && sedangProses == false) {
    window.location.href = "dashboard.html";
  }
});
```

`sedangProses` bernilai `true` selama login/daftar sedang diproses, supaya halaman tidak pindah sebelum data profil selesai disimpan.

5.3 `pesanError(kode)`

Object `daftar` berisi pasangan **kode error Firebase → pesan Bahasa Indonesia**. Jika kode ada di `daftar`, kembalikan pesannya; jika tidak, kembalikan pesan umum `"Terjadi kesalahan (kode)"`.

5.4 `setLoading(form, loading, teks)`

Menonaktifkan tombol (`disabled`) dan mengganti tulisannya jadi "Memproses..." supaya tidak diklik dua kali.

5.5 Login

```
user klik Masuk
  → e.preventDefault()            (halaman tidak reload)
  → ambil email & password
  → sedangProses = true, tombol jadi "Memproses..."
  → await signInWithEmailAndPassword(auth, email, password)
       berhasil → pindah ke dashboard.html
       gagal    → catch: sedangProses = false, tampilkan pesanError, tombol kembali "Masuk"
```

5.6 Lupa password

Tombol **"Lupa password?"** di halaman login (`#btnLupa`, `type="button"` agar tidak ikut men-submit form login):

```
user isi email lalu klik "Lupa password?"
  → email kosong? → tampilkan "Isi email dulu", return
  → tombol jadi "Mengirim..." (disabled)
  → auth.languageCode = "id"                    (email reset dalam Bahasa Indonesia)
  → await sendPasswordResetEmail(auth, email)     (Firebase yang mengirim email)
       berhasil → pesan hijau: "Jika ... terdaftar, link reset sudah dikirim. Cek Inbox dan
Spam."
       gagal    → catch: tampilkan pesanError (misal format email salah)
  → tombol kembali "Lupa password?"
```

- Pengguna membuka email, klik link-nya, lalu membuat password baru di halaman milik Firebase.
- **Kenapa pesannya "Jika ... terdaftar"?** Supaya orang lain tidak bisa menebak email mana yang punya akun. Firebase juga sengaja tidak memberi error untuk email yang tidak terdaftar.
- Email reset sering masuk **Spam** karena dikirim dari domain bawaan `firebaseapp.com`.
- Pesan sukses memakai class `pesan-sukses` (hijau); saat login, class dikembalikan ke `pesan-error` (merah).

5.7 Register

```
user klik Daftar
  → e.preventDefault()
  → ambil nama, email, password, konfirmasi
  → validasi: nama kosong? password ≠ konfirmasi?  → tampilkan pesan, return (berhenti)
  → sedangProses = true, tombol jadi "Memproses..."
  → await createUserWithEmailAndPassword(...)       → akun dibuat di Authentication
  → await setDoc(doc(db, "users", uid), {nama, email, createdAt})  → profil disimpan di
Firestore
  → pindah ke dashboard.html
```

Kenapa dua langkah? Authentication hanya menyimpan email & password. **Nama** disimpan sendiri di collection `users`, dengan **id dokumen = uid** agar mudah dicari.

Di mana password disimpan? Di Firebase Authentication, sudah otomatis di-hash. Kita tidak pernah menyimpan password di Firestore.

🧪 **Latihan:**

1. Coba login dengan password salah → lihat pesan error muncul.
2. Buka Firebase Console → Authentication → lihat user yang terdaftar. Lalu Firestore → `users` → lihat nama tersimpan.

✅ **Cek paham:** Apa fungsi `e.preventDefault()`? Kenapa pakai `if (formLogin !== null)`?

TAHAP 6 — `js/app.js` (jantung aplikasi)

File ini dibagi **10 bagian bernomor** di komentar. Pelajari **sesuai nomor**. Setiap baris hanya melakukan **satu hal**, dan percabangan memakai `if / else` biasa.

Bagian awal — Import, elemen, pengaturan, variabel global

- Import fungsi Firebase: `onAuthStateChanged`, `signOut`, `collection`, `doc`, `getDoc`, `onSnapshot`, `runTransaction`, `updateDoc`, `deleteDoc`, `serverTimestamp`.
- Ambil semua elemen HTML dengan `document.getElementById`.
- **Pengaturan (nilai tetap):**

Konstanta	Isi	Kegunaan
`PER_HALAMAN`	5	Batas tugas yang ditampilkan per halaman
`NILAI_PRIORITAS`	`{ tinggi: 1, sedang: 2, rendah: 3 }`	Mengubah prioritas jadi angka untuk diurutkan
`hariIni`	`"2026-11-09"`	Tanggal hari ini, untuk mengecek terlambat

- **Variabel global — wajib paham:**

Variabel	Isi	Kegunaan
`user`	user yang login	Ambil `user.uid` untuk menentukan lokasi tugas
`semuaTugas`	array semua tugas user	Sumber data untuk tampil, cari, filter, statistik
`editId`	`null` atau id tugas	Penanda: form sedang **tambah** atau **edit**
`halaman`	angka, mulai 1	Halaman daftar yang sedang dibuka

- **Dua fungsi alamat data** (dipakai di semua operasi database):

```
function koleksiTugas()   { return collection(db, "users", user.uid, "tasks"); }   // users/{uid}/tasks
function dokumenTugas(id) { return doc(db, "users", user.uid, "tasks", id); }      // users/{uid}/tasks/{id}
```

Artinya tugas disimpan **di dalam dokumen user pemiliknya** (subcollection), jadi di Firebase Console tugas tester dan tester2 terpisah rapi:

```
users
├── uidTester   (nama: "tester", nomorTerakhir: 12)
│   └── tasks → 0001, 0002, ... milik tester
└── uidTester2  (nama: "tester2")
    └── tasks → 0001, 0002, ... milik tester2
```

Bagian 1 — Cek login (proteksi halaman)

`onAuthStateChanged(auth, (u) ⇒ {...})` = **satpam**. Dijalankan otomatis saat halaman dibuka.

- `u === null` → belum login → `window.location.href = "index.html"`.
- `u` ada → simpan ke `user`, munculkan halaman (hapus class `memuat`), tampilkan nama, muat tugas.

`tampilkanNama()` → `getDoc` dokumen `users/{uid}`. Nama awalnya diisi email sebagai cadangan; jika dokumen ada (`snap.exists()`), ganti dengan `snap.data().nama`.

Bagian 2 — Logout

Klik tombol → `confirm()` → jika yakin: `await signOut(auth)` → pindah ke `index.html`.

Bagian 3 — READ (realtime)

```
onSnapshot(koleksiTugas(), (snapshot) ⇒ {
  semuaTugas = [];
  snapshot.docs.forEach((dokumen) ⇒ {
    const tugas = dokumen.data({ serverTimestamps: "estimate" });
    tugas.id = dokumen.id;
    if (tugas.createdAt) {
      tugas.dibuat = tugas.createdAt.toMillis();
    } else {
      tugas.dibuat = Date.now();
    }
    semuaTugas.push(tugas);
  });
  render();
});
```

Baca pelan-pelan:

1. `koleksiTugas()` → "ambil semua tugas di `users/{uid-ku}/tasks`". Tidak perlu `where`, karena folder itu memang hanya berisi tugasku.
2. `onSnapshot` → **mendengarkan** database. Setiap ada perubahan (tambah/edit/hapus), fungsi ini **jalan lagi otomatis**.

3. Kosongkan `semuaTugas`, lalu untuk **setiap dokumen**: ambil datanya, tambahkan `id` dan `dibuat` (waktu dibuat dalam angka milidetik, untuk mengurutkan), lalu `push` ke array.
4. `serverTimestamps: "estimate"` → tugas yang baru ditambah belum punya waktu dari server; pakai perkiraan dulu agar urutan tidak kacau.
5. `render()` menampilkan ulang.

> 💡 **Inilah kenapa setelah tambah/edit/hapus kita tidak memanggil render secara manual** — `onSnapshot` yang otomatis memanggilnya.

Bagian 4 — CREATE & UPDATE (satu form)

```
submit form
  → preventDefault
  → judul kosong? → toast error, return
  → deadline kosong ("")? → ubah jadi null
  → susun object data {judul, deskripsi, prioritas, kategori, deadline}
  → if (editId ≠= null) → data.updatedAt = serverTimestamp()
                          updateDoc(dokumenTugas(editId), data)      ← EDIT
    else                → tambahTugasBernomor(data)                  ← TAMBAH (ID 0001,
0002, ...)
                          halaman = 1 (agar tugas baru terlihat)
  → toast berhasil → resetForm()
  → jika gagal → catch: toast "Gagal menyimpan"
```

ID tugas berurutan — `tambahTugasBernomor(data)` (mirip `AUTO_INCREMENT` di MySQL), 5 langkah:

```
await runTransaction(db, async (tx) ⇒ {
  // 1. Baca nomor terakhir dari dokumen user (jika belum ada → 0)
  const snapUser = await tx.get(userRef);
  let nomorTerakhir = 0;
  if (snapUser.exists() && snapUser.data().nomorTerakhir) {
    nomorTerakhir = snapUser.data().nomorTerakhir;
  }
  const nomorBaru = nomorTerakhir + 1;                          // 2. tambah 1
  const idTugas = String(nomorBaru).padStart(4, "0");          // 3. 1 → "0001"
  tx.set(userRef, { nomorTerakhir: nomorBaru }, { merge: true });   // 4. simpan nomor baru
  data.nomor = nomorBaru;                                       // 5. simpan tugas dengan ID
"0001"
  data.selesai = false;
  data.createdAt = serverTimestamp();
  tx.set(dokumenTugas(idTugas), data);
});
```

- `padStart(4, "0")` → menambah angka 0 di depan sampai panjangnya 4 karakter. Lewat 9999 tetap jalan (jadi 10000).
- `runTransaction` → baca dan tulis dalam **satu paket**. Jika dua tugas ditambah bersamaan, Firebase memastikan nomornya tidak dobel.
- `{ merge: true }` → hanya mengubah field `nomorTerakhir`, field lain (nama, email) tidak terhapus.
- Nomor yang sudah dihapus **tidak dipakai ulang** (hapus 0002 → tugas berikutnya tetap 0004), sama seperti AUTO_INCREMENT.
- Nomor ini **hanya terlihat di Firestore** (sebagai ID dokumen dan field `nomor`), tidak ditampilkan di website.

Fungsi pendukung form:

- `aturModeForm(modeEdit)` → `if (modeEdit)` ubah judul form jadi "Edit Tugas", tombol "Simpan Perubahan", tampilkan tombol Batal; `else` kembalikan ke "Tambah Tugas".
- `mulaiEdit(id)` → cari tugas dengan `find`, isi kelima input form dengan datanya, set `editId = id`, panggil `aturModeForm(true)`.
- `resetForm()` → `formTugas.reset()` (prioritas otomatis kembali ke "Sedang" karena `selected` di HTML), `editId = null`, `aturModeForm(false)`.

Bagian 5 — DELETE dengan Sampah (soft delete)

Tugas yang terhapus tidak langsung hilang, tapi masuk 🗑️ **Sampah** dulu — seperti Recycle Bin di Windows.

Tombol	Fungsi	Yang terjadi di Firestore
Hapus (di daftar biasa)	`hapusTugas(id)`	`updateDoc` → `dihapus: true`, `dihapusPada: waktu`. **Dokumen masih ada**
Pulihkan (di Sampah)	`pulihkanTugas(id)`	`updateDoc` → `dihapus: false`. Tugas kembali ke daftar
Hapus Permanen (di Sampah)	`hapusPermanen(id)`	`confirm()` lalu `deleteDoc` → **dokumen benar-benar hilang**

- `diSampah(t)` → `return t.dihapus === true;`. Tugas lama yang belum punya field `dihapus` dianggap **tidak** di sampah.
- Membuka Sampah: pilih 🗑️ **Sampah** di dropdown status. Jumlah isinya tampil di dropdown, contoh "🗑️ Sampah (3)".
- Di Sampah, tombolnya berganti menjadi **Pulihkan** dan **Hapus Permanen**, kartu bergaris putus-putus, dan tombol centang disembunyikan.
- Hanya **Hapus Permanen** yang memakai `confirm()`, karena hanya aksi itu yang tidak bisa dibatalkan.
- Istilah ini disebut **soft delete** (hapus lunak) vs **hard delete** (hapus permanen).

Bagian 6 — Status tugas: Belum → Sedang dikerjakan → Selesai

Setiap tugas punya dua field status: `selesai` dan `dikerjakan` (keduanya `true` / `false`).

Aksi pengguna	Fungsi	Yang disimpan di Firestore
Klik judul/deskripsi tugas (atau tekan Enter)	`toggleDikerjakan(id, statusSekarang)`	`dikerjakan: !statusSekarang` → mulai / batal dikerjakan
Klik tombol hijau **Selesai** (muncul saat sedang dikerjakan)	`tandaiSelesai(id)`	`selesai: true`, `dikerjakan: false`
Klik tombol **Batal selesai** (muncul pada tugas yang sudah selesai)	`toggleSelesai(id, true)`	`selesai: false`, `dikerjakan: false`

Kotak centang di kiri hanya penanda status, tidak bisa diklik manual (elemen `<span>`, bukan `<button>`). Kotak itu otomatis tercentang setelah tombol **Selesai** diklik. Jadi alurnya selalu: klik tugas → Sedang dikerjakan → Selesai.

```
await updateDoc(dokumenTugas(id), {
  selesai: !statusSekarang,   // ! membalik: false → true, true → false
  dikerjakan: false,
  updatedAt: serverTimestamp()
});
```

- `sedangDikerjakan(t)` → `return t.dikerjakan === true && t.selesai === false;`. Tugas lama tanpa field `dikerjakan` dianggap tidak sedang dikerjakan.
- Tugas yang sedang dikerjakan: latar biru muda, badge "⏳ **Sedang dikerjakan**", kotak centang bergaris putus-putus, dan ada tombol **Selesai**.
- Tugas yang sudah selesai atau ada di sampah **tidak bisa diklik** untuk dikerjakan.
- Agar bisa dipakai dengan keyboard, area tugas diberi `tabIndex = 0` dan `role="button"`, dan tombol **Enter** menjalankan fungsi yang sama dengan klik.

Bagian 7 — Cari + filter + urutkan

```
function ambilTugasTerfilter() {
  const hasil = semuaTugas.filter(lolosFilter);   // 1. saring
  hasil.sort(bandingkan);                         // 2. urutkan
  return hasil;
}
```

`lolosFilter(t)` — mengembalikan `true` jika tugas boleh tampil. Dicek satu per satu, begitu ada yang tidak cocok langsung `return false`:

1. Judul tidak mengandung kata cari → `false` (`toLowerCase` agar huruf besar/kecil tidak berpengaruh).
2. Sampah: jika dropdown = "Sampah" → tampilkan **hanya** tugas di sampah (`return diSampah(t)`). Selain itu, tugas di sampah selalu disembunyikan.
3. Filter "selesai" tapi tugas belum selesai → `false` . Filter "belum" tapi tugas sudah selesai → `false` . Filter "Sedang dikerjakan" tapi `sedangDikerjakan(t)` bernilai `false` → `false` .
4. Filter kategori bukan "semua" dan kategori tugas berbeda → `false` .
5. Lolos semua → `true` .

`bandingkan(a, b)` — hasil negatif = `a` di atas, positif = `b` di atas:

1. Tugas belum selesai **selalu** di atas tugas selesai.
2. Tugas yang **sedang dikerjakan** di atas tugas lain yang belum selesai.
3. Sesuai dropdown **Urutkan** (pakai `if / else if`):
   - *Terbaru* → `b.dibuat - a.dibuat` (yang baru dibuat di atas).
   - *Terlama* → `a.dibuat - b.dibuat` .
   - *Prioritas* → `NILAI_PRIORITAS[a.prioritas] - NILAI_PRIORITAS[b.prioritas]` , angka kecil di atas.
   - *Deadline* → `bandingkanDeadline(a, b)` : deadline terdekat dulu; tugas tanpa deadline dianggap `"9999-12-31"` sehingga paling bawah.
4. Jika hasilnya 0 (seri) → urut sesuai **waktu dibuat**.

> 🐞 **Cerita bug (bagus diceritakan ke penguji):** versi awal hanya mengurutkan berdasarkan deadline. Saat semua tugas tidak punya deadline, nilainya sama semua, sehingga urutannya mengikuti ID acak Firestore → tampil acak (7, 6, 8, 1, 3…). Solusinya: tambah aturan ke-3, *jika seri, urutkan berdasarkan waktu dibuat*.

`gantiFilter()` dipanggil setiap mengetik di kotak cari (event `input`) atau mengganti dropdown (event `change`): kembali ke `halaman = 1` lalu `render()` . **Kenapa di JavaScript, bukan query Firestore?** Lebih sederhana, cepat (data sudah ada di memori), dan tidak perlu membuat index di Firestore.

Bagian 8 — Statistik

```
tugasAktif = semuaTugas tanpa yang di sampah   (sampah tidak dihitung)
total     = tugasAktif.length
selesai   = jumlah tugas yang selesai
belum     = total - selesai
terlambat = jumlah tugas yang cekTerlambat(t) === true
persen    = 0, lalu jika total > 0 → Math.round(selesai / total * 100)   (cegah bagi nol)
```

Setiap angka ditulis ke kartu statistiknya masing-masing dengan `textContent` , dan lebar progress bar `style.width = persen + "%"` .

`cekTerlambat(t)` :

1. Sudah selesai → bukan terlambat.
2. Tidak punya deadline → bukan terlambat.
3. Selain itu → terlambat jika `t.deadline < hariIni`. Tanggal format `YYYY-MM-DD` bisa dibandingkan langsung seperti teks.

Bagian 9 — Render (menampilkan daftar + paginasi)

`render()` :

1. Hitung statistik.
2. Ambil daftar terfilter (`list`).
3. Kosongkan `#daftarTugas` dan `#paginasi` .
4. Jika `list` kosong → tampilkan pesan "Belum ada tugas" / "Tidak ada yang cocok", lalu berhenti.
5. **Paginasi** (batas 5 tugas per halaman):

```
const totalHalaman = Math.ceil(list.length / PER_HALAMAN);   // 12 tugas → 3 halaman
if (halaman > totalHalaman) {
  halaman = totalHalaman;                                     // misal setelah tugas
terakhir dihapus
}
const awal = (halaman - 1) * PER_HALAMAN;                     // hal 1 → 0, hal 2 → 5, hal
3 → 10
const potongan = list.slice(awal, awal + PER_HALAMAN);        // ambil 5 tugas saja
```

6. Perulangan `for` → setiap tugas di `potongan` dibuat kartunya dengan `buatItemTugas` , lalu `appendChild` .
7. Jika lebih dari 1 halaman → `buatPaginasi()` membuat tombol **« Sebelumnya | 1 2 3 | Berikutnya »**. Tombol halaman aktif diberi class `aktif` ; Sebelumnya nonaktif di halaman 1, Berikutnya nonaktif di halaman terakhir.

`buatElemen(tag, kelas, teks)` — fungsi bantu untuk membuat elemen HTML + class + teks. Dipakai di semua tempat yang membuat elemen, sehingga tidak perlu menulis `createElement` , `className` , `textContent` berulang-ulang. Memakai **`textContent`** → aman dari XSS (kode `<script>` tampil sebagai teks biasa).

`buatItemTugas(t)` membuat satu kartu tugas, urut dari atas:

1. Kartu `div.tugas` + class prioritas, ditambah class `selesai` jika selesai.
2. Kotak centang (`<span>` , hanya penanda): berisi ✓ jika selesai; `aria-label` berisi status tugas untuk pembaca layar.
3. Isi: judul (`h3`) dan deskripsi (jika ada).

4. Badge, masing-masing dengan `if`: Terlambat / Hari ini, prioritas, kategori, tanggal (jika ada), Selesai (jika selesai).
5. Tombol Edit → `mulaiEdit`, tombol Hapus → `hapusTugas`.
6. Semua bagian dimasukkan ke kartu dengan `appendChild`.

`buatBadge(teks, kelas)` → membuat `<span class="badge ...">`. `formatTanggal("2026-11-20")` → `"20 Nov 2026"`.

Bagian 10 — Toast

Isi teks → beri class `toast tampil` (atau `toast tampil error` jika error) → `setTimeout` 2,5 detik → hapus class `tampil` (hilang). `clearTimeout` mencegah toast lama mengganggu yang baru.

🧪 **Latihan Tahap 6:**

1. Tambahkan `console.log(semuaTugas);` di dalam `onSnapshot`, buka Console (F12), lalu tambah/hapus tugas → lihat array berubah otomatis. (Hapus lagi setelah selesai.)
2. Tambah tugas, lalu lihat dokumennya di Firebase Console → Firestore → `users` → klik uid-mu → `tasks`. Cocokkan field-nya.
3. Tambah 6 tugas atau lebih → muncul tombol halaman. Coba pindah halaman dan ganti dropdown Urutkan.

✅ **Cek paham:**

- Apa beda `tambahTugasBernomor` (pakai `tx.set`) dan `updateDoc`?
- Bagaimana satu form bisa untuk tambah dan edit?
- Kenapa setelah menambah tugas tidak perlu memanggil `render()`?
- Apa fungsi `buatElemen`, dan kenapa memakai `textContent`, bukan `innerHTML`?

TAHAP 7 — `firestore.rules` (keamanan)

```
pemilik(userId)  → login DAN uid yang login == userId di alamat data
tugasValid(d)    → judul teks 1-100 huruf, prioritas & kategori sesuai pilihan, selesai boolean

users/{userId}                 → baca/tulis jika pemilik
users/{userId}/tasks/{taskId}  → baca/hapus jika pemilik
                                 tambah/edit jika pemilik DAN tugasValid
```

- `request.auth` = user yang sedang login (null jika belum login).
- `request.resource.data` = data **baru** yang akan disimpan (dicek oleh `tugasValid`).
- Karena tugas ada di dalam `users/{userId}`, cukup cocokkan `userId` di alamat dengan uid yang login.

Kenapa perlu, padahal di JS sudah memakai `user.uid`? Kode JavaScript bisa diubah orang lewat browser. Rules dijalankan di **server Firebase**, jadi tidak bisa diakali — termasuk mencegah data aneh (judul kosong, prioritas "super") masuk ke database.

✅ **Cek paham:** Kenapa tugas tester2 tidak bisa dibaca tester? Apa fungsi `tugasValid`?

TAHAP 8 — Latihan Modifikasi (bukti benar-benar paham)

Kerjakan satu per satu. Jika bisa, berarti kamu paham alurnya. (Lakukan di **salinan folder**, jangan di versi ujian.)

No	Tantangan	Petunjuk file/bagian
1	Tambah kategori baru "Hobi"	`dashboard.html` : dua `<select>` (form & filter)
2	Ubah lama toast jadi 4 detik	`app.js` bagian 10 (`2500`)
2b	Ubah batas per halaman jadi 10	`app.js` variabel `PER_HALAMAN`
3	Ubah warna utama jadi hijau	`style.css` `--tinta`
4	Judul minimal 3 huruf	`app.js` bagian 4 (`judul.length < 3`)
5	Tampilkan jumlah tugas prioritas tinggi di console	bagian 8: `semuaTugas.filter(t ⇒ t.prioritas === "tinggi").length`
6	Tombol "Hapus semua yang selesai"	loop `semuaTugas.filter(t ⇒ t.selesai)` → `deleteDoc` setiap id
7	Badge "Besok" untuk deadline besok	bagian 9, hitung tanggal besok seperti `hariIni`

Penguji sering meminta **mengubah sedikit kode secara langsung**, jadi latihan ini sangat penting.

TAHAP 9 — Persiapan Presentasi

9.1 Urutan demo (± 5 menit)

1. **Perkenalan** (30 detik): kalimat di Tahap 0.4.
2. **Struktur folder** (30 detik): tunjukkan 3 HTML, 1 CSS, 3 JS, rules.
3. **Demo register** → tunjukkan user baru di Firebase Authentication & `users`.
4. **Demo login** salah lalu benar → tunjukkan pesan error.
5. **Demo CRUD:** tambah 2 tugas → buka Firestore tunjukkan dokumennya → edit → centang selesai (statistik berubah) → hapus.
6. **Demo cari & filter.**
7. **Demo keamanan:** buka `dashboard.html` setelah logout → terlempar ke login; ketik `<script>alert(1)</script>` sebagai judul → tampil sebagai teks.
8. **Jelaskan kode** bila diminta: mulai dari `firebase.js` → `auth.js` → `app.js` bagian 1 sampai 10.

9.2 Teknik menjelaskan kode (rumus 3 langkah)

Untuk setiap bagian, jawab: **"Ini untuk apa → bagaimana caranya → hasilnya apa."** Contoh untuk toggle:

> *"Ini untuk menandai tugas selesai. Caranya memakai `updateDoc` ke dokumen tugas, field `selesai` diisi kebalikan nilai sekarang dengan tanda `!`. Hasilnya data di Firestore berubah, `onSnapshot` mendeteksi, dan tampilan otomatis diperbarui — judul jadi tercoret."*

9.3 Kuis mandiri (tutup jawaban, jawab lisan)

No	Pertanyaan	Jawaban
1	Sebutkan bahasa yang dipakai & fungsinya	HTML struktur, CSS tampilan, JS logika; Firestore database
2	Firestore SQL atau NoSQL?	NoSQL — collection, document, field
3	Struktur database?	`users/{uid}` (profil) dan subcollection `users/{uid}/tasks` (tugas milik user itu)
4	Field di tugas?	nomor, judul, deskripsi, prioritas, kategori, deadline, selesai, dihapus, dihapusPada, createdAt, updatedAt (ID dokumen: 0001, 0002, ...)
5	Penghubung user dan tugas?	Tugas disimpan di dalam dokumen user-nya (subcollection) → one-to-many, rapi per akun
6	Fungsi CRUD Firestore yang dipakai?	Create `runTransaction` + `tx.set`, Read `onSnapshot` / `getDoc`, Update `updateDoc`, Delete `deleteDoc`
7	Cara mencegah user belum login masuk dashboard?	`onAuthStateChanged`; jika user null → redirect ke index.html
8	Kenapa `async/await`?	Akses Firebase butuh waktu; `await` menunggu hasil sebelum lanjut
9	Fungsi `try/catch`?	Menangkap error agar aplikasi tidak berhenti & bisa tampilkan pesan
10	Apa itu `onSnapshot`?	Pendengar realtime; tampilan otomatis update saat data berubah
11	Bagaimana tambah & edit pakai satu form?	Variabel `editId`: null = tambah (`tambahTugasBernomor`), ada isi = edit (`updateDoc`); tampilan form diatur `aturModeForm`
12	Bagaimana cari & filter bekerja?	`.filter()` berantai pada array `semuaTugas`, lalu `render()`
13	Bagaimana menghitung terlambat?	belum selesai && ada deadline && deadline < hariIni
14	Bagaimana mencegah XSS?	Menampilkan data user dengan `textContent`
15	Bagaimana data user lain aman?	Tugas di path `users/{uid}/tasks` + Security Rules memastikan hanya pemilik uid yang boleh akses
16	Kenapa harus Live Server?	`type="module"` tidak bisa jalan dari file:// (diblokir browser)
17	Fungsi `e.preventDefault()`?	Mencegah form me-reload halaman

No	Pertanyaan	Jawaban
18	Fungsi media query?	Menyesuaikan tampilan untuk layar HP
19	Kekurangan Firebase?	Butuh internet; bergantung pada layanan Google
20	Kenapa password tidak ada di Firestore?	Diurus Firebase Authentication, otomatis di-hash
21	Bagaimana paginasi bekerja?	List dipotong `slice(awal, awal + 5)`, `awal = (halaman - 1) × 5`, jumlah halaman `Math.ceil(total / 5)`
22	Kenapa dulu urutan tugas acak?	Semua tanpa deadline → nilai sama → ikut urutan ID acak Firestore. Diperbaiki dengan urutan cadangan berdasarkan waktu dibuat
23	Bagaimana ID tugas bisa 0001, 0002?	Nomor terakhir disimpan di `users/{uid}.nomorTerakhir`; saat tambah, dibaca +1 dalam `runTransaction`, lalu `padStart(4,"0")` dijadikan ID dokumen
24	Kenapa ada fungsi bantu seperti `buatElemen`, `aturModeForm`, `setLoading`?	Prinsip DRY (Don't Repeat Yourself): kode yang sama ditulis sekali sebagai fungsi, lalu dipakai berulang → tidak perlu menulis ulang & mudah diperbaiki
25	Bagaimana fitur Sampah bekerja?	Soft delete: Hapus hanya mengubah field `dihapus: true` (data tetap di Firestore). Pulihkan → `dihapus: false`. Hapus Permanen → `deleteDoc` (data hilang)
26	Bagaimana fitur Lupa password bekerja?	`sendPasswordResetEmail` → Firebase mengirim link reset ke email; pengguna membuat password baru lewat link itu. Password lama tidak pernah bisa dilihat karena disimpan dalam bentuk hash
27	Bagaimana status "Sedang dikerjakan" bekerja?	Klik tugas → `toggleDikerjakan` menyimpan `dikerjakan: true`. Tombol Selesai → `tandaiSelesai` menyimpan `selesai: true, dikerjakan: false`. Tugas yang sedang dikerjakan selalu tampil paling atas

9.4 Ujian terakhir: ketik ulang dari nol

Di folder baru, coba ketik ulang `app.js` **bagian 3–6** (read, create/update, delete, toggle) tanpa melihat. Jika macet, lihat panduan ini (bukan kodenya), lalu coba lagi. Lakukan sampai lancar — ini bekal jika penguji meminta membuat fitur di tempat.

Checklist Akhir

- [ ] Bisa menggambar alur aplikasi (Tahap 0.3) tanpa melihat.
- [ ] Bisa menjelaskan fungsi setiap file dalam 1 kalimat.
- [ ] Bisa menjelaskan login & register langkah demi langkah.
- [ ] Bisa menjelaskan 10 bagian `app.js` dengan rumus "untuk apa → caranya → hasilnya".
- [ ] Bisa menunjukkan data di Firebase Console.
- [ ] Menyelesaikan minimal 5 dari 7 latihan modifikasi.
- [ ] Menjawab 20 kuis tanpa melihat jawaban.
- [ ] Sudah latihan demo minimal 2 kali (pakai timer 5 menit).